



Student Name: Bhavya
Branch: CSE(AI & ML)
Semester: 4
Subject Name: Database Management System

UID: 24BAI70791
Section/Group: 24AIT_KRG-1/G2
Date of Performance: 27/02/2026
Subject Code: 24CSH-298

Experiment – 5

1. Aim of the Session

To understand and apply **conditional logic in SQL** by using the **modulus operator (MOD / %)** to analyze numerical data and classify employee salaries as **odd or even**, thereby improving data analysis and decision-making skills in SQL.

2. Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

3. Objectives

To write and execute SQL queries that use the **MOD (%) operator** to check employee salaries and display **odd and even salary values separately** from an employee table.

4. Procedure of the Experiment

1. Create an **employee** table with fields emp_id, emp_name, and salary.
2. Insert sample employee data into the table using the INSERT command.
3. Display all records from the employee table using the SELECT statement.
4. Use the MOD (%) operator to check and separate even and odd salary values.
5. Use the CASE statement to label each salary as “Even” or “Odd” and observe the output.

5. Practical / Experiment Steps

PROGRAM CODE:

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO employee VALUES (101, 'Rahul', 25000);
```

```
INSERT INTO employee VALUES (102, 'Neha', 30001);
```

```
INSERT INTO employee VALUES (103, 'Amit', 18000);
```

```
INSERT INTO employee VALUES (104, 'Priya', 27555);
```

```
INSERT INTO employee VALUES (105, 'Karan', 40000);
```

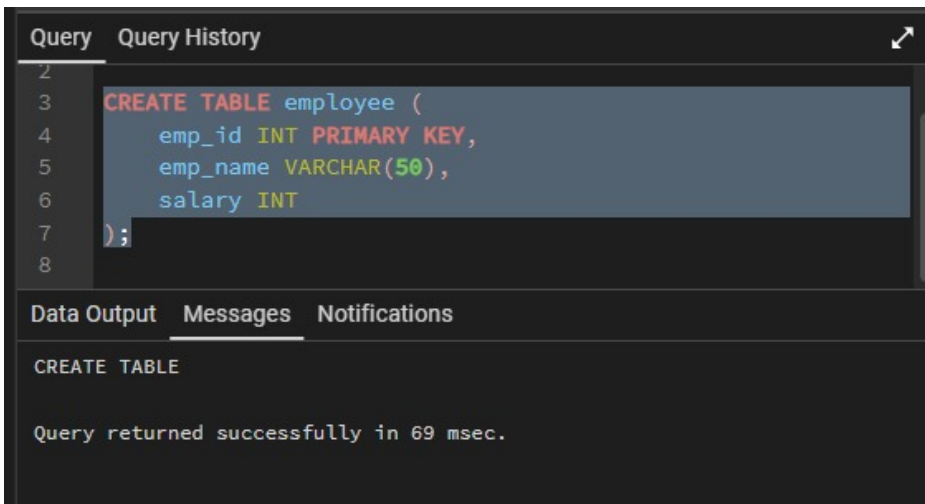
```
SELECT * FROM employee;
```

```
SELECT emp_id, emp_name, salary  
FROM employee  
WHERE MOD(salary, 2) = 0;
```

```
SELECT emp_id, emp_name, salary  
FROM employee  
WHERE MOD(salary, 2) = 1;
```

```
SELECT emp_id, emp_name, salary,  
CASE  
    WHEN MOD(salary, 2) = 0 THEN 'Even Salary'  
    ELSE 'Odd Salary'  
END AS salary_type  
FROM employee;
```

6. Input / Output Details and Screenshot



The screenshot shows a SQL IDE interface with a dark theme. The top bar has tabs for 'Query' and 'Query History'. The main editor area displays the following SQL code:

```
2  
3 CREATE TABLE employee (  
4     emp_id INT PRIMARY KEY,  
5     emp_name VARCHAR(50),  
6     salary INT  
7 );  
8
```

Below the editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output: 'CREATE TABLE' and 'Query returned successfully in 69 msec.'

Query Query History

```

1 CREATE TABLE employee (
2     emp_id INT PRIMARY KEY,
3     emp_name VARCHAR(50),
4     salary INT
5 );
6
7 INSERT INTO employee VALUES (101, 'Rahul', 25000);
8 INSERT INTO employee VALUES (102, 'Neha', 30001);
9 INSERT INTO employee VALUES (103, 'Amit', 18000);
10 INSERT INTO employee VALUES (104, 'Priya', 27555);
11 INSERT INTO employee VALUES (105, 'Karan', 40000);
12

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 62 msec.

```

12
13 SELECT * FROM employee;
14
15

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	salary integer
1	101	Rahul	25000
2	102	Neha	30001
3	103	Amit	18000
4	104	Priya	27555
5	105	Karan	40000

```

15 SELECT emp_id, emp_name, salary
16 FROM employee
17 WHERE MOD(salary, 2) = 0;
18
19
20

```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	salary integer
1	101	Rahul	25000
2	103	Amit	18000
3	105	Karan	40000

```

19 SELECT emp_id, emp_name, salary
20 FROM employee
21 WHERE MOD(salary, 2) = 1;
22

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	salary integer
1	102	Neha	30001
2	104	Priya	27555

```

23 SELECT emp_id, emp_name, salary,
24 CASE
25     WHEN MOD(salary, 2) = 0 THEN 'Even Salary'
26     ELSE 'Odd Salary'
27 END AS salary_type
28 FROM employee;
29

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	salary integer	salary_type text
1	101	Rahul	25000	Even Salary
2	102	Neha	30001	Odd Salary
3	103	Amit	18000	Even Salary
4	104	Priya	27555	Odd Salary
5	105	Karan	40000	Even Salary

7. Learning Outcome

- Understand the use of the **MOD (%) operator** in SQL.
- Apply **conditional logic** to classify numerical data as odd or even.
- Write and execute **SELECT queries with conditions** using the WHERE clause.
- Use the **CASE statement** to generate meaningful labels in query results.
- Improve skills in **data analysis and decision-making** using SQL queries.